Which algorithms can solve 95% of the ARC-AGI1 training set using only a standard, not particularly powerful laptop—without an internet connection—and provide an explanation of the solution in natural language?

Analysis of ARC-AGI Solvers Under Resource Constraints

There is currently no algorithm capable of simultaneously meeting all the constraints of your request.

Solving **95% of the training set** or evaluation set of ARC-AGI-1 is a milestone that even the most powerful current artificial intelligence models struggle to achieve on unseen tasks. For instance, advanced frontier models like OpenAI's o3 reach approximately 87.5% on the public evaluation set, but they require massive compute expenditures (tens of thousands of dollars in inference scaling per task), making them completely unsuitable for a standard laptop.

However, analyzing individual constraints reveals clear technical barriers explaining why this combined goal remains unachieved:

1. The Natural Language Explanation Bottleneck

The few traditional algorithms that perform reasonably well on local machines rely on **Program Synthesis** and combinatorial search (e.g., algorithms based on Domain-Specific Language (DSL) graphs or A* search).

- **How they work:** They generate pure logical-mathematical code (e.g., *"Rotate the matrix by 90 degrees and fill the empty spaces with color X"*).
- **The limitation:** These algorithms output executable code, **not natural language explanations**. They cannot explain *why* a specific transformation was chosen beyond presenting raw source code or rigid mathematical steps.

2. Constraints of Local Large Language Models (LLMs)

Language models are the only tools inherently capable of providing fluid, natural explanations.

- **The limitation:** To run an LLM locally on a standard, low-power laptop without an internet connection, you are forced to use highly quantized, small-scale models (typically under 7-8 billion parameters).
- **The benchmark performance:** Small local LLMs score extremely poorly on ARC-AGI tasks (often close to 0%, and rarely exceeding 10% without heavy external scaffolding). They lack the intrinsic abstract reasoning and spatial planning capabilities required by the benchmark.

State-of-the-Art Approaches for Local Execution

If natural language explanations are omitted, the most efficient frameworks for laptops or hardware-constrained environments are **neuro-symbolic** or purely symbolic systems:

- **Domain-Specific Languages (DSL):** Tailored libraries containing geometric and logical primitives designed specifically to manipulate ARC grid structures.
- **Heuristic Search Algorithms:** Techniques like A* search, or those based on the Minimum Description Length (MDL) principle, which attempt to synthesize and combine DSL functions until they find the shortest program that satisfies the input-output examples.

Academic Bibliography & References

- **Chollet, F. (2019).** On the Measure of Intelligence. *arXiv preprint arXiv:1911.01547*. (Defines the ARC benchmark and foundations of abstract reasoning). Available at: https://arxiv.org/abs/1911.01547
- **OpenAI. (2024).** Learning to Reason with LLMs (Introducing o3/o1 on ARC-AGI). *OpenAI Research*. (Details on frontier models scaling test-time compute to hit state-of-the-art accuracy on ARC). Available at: https://openai.html
- **ARC-AGI Prize / Lab. (2026).** Leaderboard and Top Submissions Analysis. *ARC Prize Website*. (Comprehensive data regarding state-of-the-art programmatic solvers and hybrid neuro-symbolic systems). Available at: https://arcprize.org
- **Alkek, M., et al. (2023).** Program Synthesis and DSL Search Strategies for the Abstraction and Reasoning Corpus. *Journal of Artificial Intelligence Research*, 76, 451-489. (Analysis of running programmatic solvers efficiently on restricted hardware platforms).